

SRLR: Symbolic Regression-Based Logic Recovery to Counter Programmable Logic Controller Attacks

Hao Zhou¹, Suman Sourav², Binbin Chen³, *Member, IEEE*, and Ke Yu¹

Abstract—Programmable Logic Controllers (PLCs) are critical components in Industrial Control Systems (ICSs). Their potential exposure to external world makes them susceptible to cyber-attacks. Existing detection methods against controller logic attacks use either specification-based or learnt models. However, specification-based models require experts' manual efforts or access to PLC's source code, while machine learning-based models often fall short of providing explanation for their decisions. We design *SRLR* — a *Symbolic Regression based Logic Recovery* solution to identify the logic of a PLC based only on its inputs and outputs. The recovered logic is used to generate explainable rules for detecting controller logic attacks. *SRLR* enhances the latest deep symbolic regression methods using the following ICS-specific properties: 1) some important ICS control logic is best represented in frequency domain rather than time domain; 2) an ICS controller can operate in multiple modes, each using different logic, where mode switches usually do not happen frequently; 3) a robust controller usually filters out outlier inputs as ICS sensor data can be noisy; and 4) with the above factors captured, the degree of complexity of the formulas is reduced, making effective search possible. Thanks to these enhancements, *SRLR* consistently outperforms all existing methods in a variety of ICS settings that we evaluate. In terms of the recovery accuracy, *SRLR*'s gain can be as high as 39% in some challenging environment. We also evaluate *SRLR* on a distribution grid containing hundreds of voltage regulators, demonstrating its stability in handling large-scale, complex systems with varied configurations.

Index Terms—Cyber-physical system security, symbolic regression, logic recovery, programmable logic controller.

I. INTRODUCTION

IN THE era of Industry 4.0, critical infrastructures are integrated with advanced communication networks and physical processes, leading to the emergence of diverse ICSs such as water treatment plants and power control systems.

Received 23 October 2024; revised 1 June 2025 and 10 August 2025; accepted 4 November 2025. Date of publication 17 November 2025; date of current version 26 November 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62371057; and in part by the National Research Foundation, Singapore, under its National Satellite of Excellence Program “Design Science and Technology for Secure Critical Infrastructure: Phase II” under Award NRF-NCR25-NSOE05-0001. The associate editor coordinating the review of this article and approving it for publication was Dr. Daisuke Mashima. (*Corresponding author: Ke Yu.*)

Hao Zhou and Ke Yu are with Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: zhouh@bupt.edu.cn; yuke@bupt.edu.cn).

Suman Sourav is with Aalborg University, 9220 Aalborg, Denmark (e-mail: sumansourav@cs.aau.dk).

Binbin Chen is with Singapore University of Technology and Design, Singapore 487372 (e-mail: binbin_chen@sutd.edu.sg).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TIFS.2025.3634027>, provided by the authors.

Digital Object Identifier 10.1109/TIFS.2025.3634027

However, this integration in ICSs exposes industrial process to various vulnerabilities, thus precipitating severe security challenges [1], [2]. Internet-connected PLCs play a pivotal role in ICSs. High-profile attacks, such as Stuxnet [3], manipulate the configuration of PLCs or tamper with PLC's output commands to compromise physical processes.

Various Intrusion Detection Systems (IDSs) have been developed to detect malicious activities in ICSs. Many of them focus on detecting manipulated PLC logic. State-of-the-art (SOTA) deep learning-based IDSs [4], [5], [6] gather sensor and actuator readings within ICSs to construct profiles for normal or anomalous data. However, these techniques are black-boxes where the reasoning behind the detection results is non-explainable, even for domain experts. In contrast, specification-based IDSs [7], are explainable but rely on rules derived from expert knowledge. Crafting and maintaining these manual rules is labor-intensive, error-prone, and difficult. Recent advancements (e.g., [8]) include the design of automatic rule generators by parsing PLCs' code (which can thereby be used by an IDS); however, these codes are often proprietary in nature and inaccessible to third-party security vendors. Specifically, based on our first-hand experiences with several state-owned ICS operators, their PLC code is classified as “CONFIDENTIAL” by regulation, so their policy strictly prohibits sharing the code outside their organizations, even to contracted security-vendors; moreover, some core PLC logic is the intellectual-property of solution-vendors and is not accessible even to the ICS operators themselves. Consequently, without access to the logical information of PLCs, these methods encounter significant challenges in effectively addressing complex attack detection problems in real-world ICS environments. On the other hand, ICS operators are usually open for contracted security-vendors to instrument their system in a non-invasive way, i.e. allowing security-vendors to passively gather input/output data of PLCs. Additionally, high-fidelity in-house testing facilities (running the same PLC logic as in the field) are usually provided for security-vendors to calibrate their solutions for pre-deployment testing. In this regard, there is a need for accurate attack detection that (1) doesn't rely on code availability from PLC vendors; (2) instead, leverages available system log and network trace; and (3) provides transparent and explainable detection decisions.

In this work, we propose *SRLR* (*Symbolic Regression based Logic Recovery*) — a method designed to utilize PLCs' inputs (sensor readings) and outputs (actuator commands) to identify PLCs' key control logic and generate a series of rules for attack detection. The design of *SRLR* is feasible because:

TABLE I
SUMMARY OF COMPARISONS WITH DIFFERENT METHODS

Logic recovery models	Time domain		Frequency domain	
	Single-mode logic	Multi-mode logic	Single-mode logic	Multi-mode logic
NARMAX, NARX, FIR [9]	✓	×	✓	×
Genetic Programming [10], [11]	✓	×	×	×
Cluster SR [12]	✓	✓	×	×
SRLR (ours)	✓	✓	✓	✓
Attack detection models	Explainable with human-understandable rules		Minimal prior knowledge without knowledge of attack signatures	
			Easy installation without access to PLC code	
Specification based methods [1], [13]	✓	×	×	×
Machine Learning (ML) based methods [4], [5], [14], [15]	×	✓	✓	✓
CoToRu [8]	✓	✓	✓	×
SRLR (ours)	✓	✓	✓	✓

For many PLCs, the initiation of their control commands is contingent upon some key decision variables, whose values are determined by mathematical equations of input sensor readings and governed by physical laws and control loop mechanisms. While there are complicated PLC logic (e.g., state-based logic or those with dozens of conditional branches) that do not offer succinct mathematical representation (hence, beyond the scope of our work), we believe that by recovering PLC logic that can be expressed as mathematically deterministic equations between PLC inputs and outputs, one can already cover a large subset of PLC control behaviors in many domains — from power systems to other important industrial control systems, as we will validate through our experiments. This observation inspires us to employ Deep Symbolic Regression (DSR) [16] to infer the formulaic mappings, wherein PLCs' outputs are expressed as mathematical functions of sensor readings. SRLR enhances the design of DSR based on the following ICS-unique properties: (1) For some important ICS domains, the control logic to be recovered often has specific patterns and expected level of complexity (e.g., including integral and differentiation components), and the simplest representation of such logic is not in the time domain, but in the frequency domain. One of our contributions in designing SRLR is to use such domain knowledge and map the original time-domain data into the complex-valued frequency domain, or known as s-domain, before recovering the control logic. (2) A PLC may be configured to operate in multiple modes, with each mode running different logic. The mode switches in ICS are not frequent. This observation inspires us to infer multi-mode logic using a continuity constraint to comprehensively profile the PLCs' control logic. (3) In ICSs, the sensor readings that are fed into PLCs often contain outliers or noise. To deal with that, a robust PLC usually has built-in logic to filter out such outliers. We also design an outlier-aware training algorithm for SRLR to prevent outlier data from driving the search in the wrong direction. (4) Lastly, we propose an effective regularization method to guide the search towards the appropriate complexity level.

Thanks to the above enhancements, SRLR is able to recover the controller logic with higher accuracy compared to the DSR method. The derived formulas by SRLR can be leveraged to formulate invariant rules that describe the normal operational profile of PLCs. Deviation from these rules can then indicate in an explainable manner anomalous behavior observed in the PLCs' operation. We summarize the difference between SRLR and the main SOTA methods in Table I.

Using the control logic recovered by SRLR, we generate a series of rules that accurately describe the normal operational profile of PLCs. Deviation from these rules can be used to detect PLC attacks. To show the effectiveness of SRLR in detecting controller logic attacks, we extensively evaluate SRLR using a variety of ICSs in different settings. We show that SRLR outperforms all the existing methods by up to 39% in terms of accuracy. For the Secure Water Treatment (SWaT) dataset [17], SRLR not only improves the attack detection accuracy on recent SOTA schemes, and it also makes the decision more explainable by mapping to the corresponding invariant that is being violated.

II. BACKGROUND AND THREAT MODEL

A. Logic Recovery

There are industrial processes that operate under a singular mode (or state) with static control logic. We define the recovery of such control logic as a *single-mode logic recovery*. Making it more challenging, PLCs in many ICSs encompass multi-mode logic, which carry out different logic based on state-transition conditions. We refer to the problem of profiling the switched control logics, i.e., recovering the control logic for each operating mode/configuration of the considered system as *multi-mode logic recovery*. We assume in each mode (be it single or multi-mode system), there is a succinct mathematical equation that governs the relationship between the output of a PLC and its inputs.

One fundamental concept we use to recover logic is symbolic regression (SR). SR has been extensively studied in the field of discovering physical laws [18] aiming to derive mathematical formulas that accurately represent data generated by physical systems. Given a dataset (X, y) , where $X_i \in \mathbb{R}^n, y_i \in \mathbb{R}$, the objective of SR is to identify the function $f: \mathbb{R}^n \rightarrow \mathbb{R}$, such that $y_i = f(X_i)$ for each data point.

In this work, we adopt the recent SOTA model, namely DSR [16], as the backbone framework for identifying control logic. As illustrated in Figure 1, DSR operates within a reinforcement learning framework, where the Recurrent Neural Network (RNN) serves as the policy. The symbolic representations act as the state, while emitting the probabilities of each symbol within a library (e.g., {+, −, etc.}) constitutes the action. During an episode, the sampled symbolic sequence can construct an expression tree, representing a mathematical expression τ through a pre-order traversal. The risk-policy

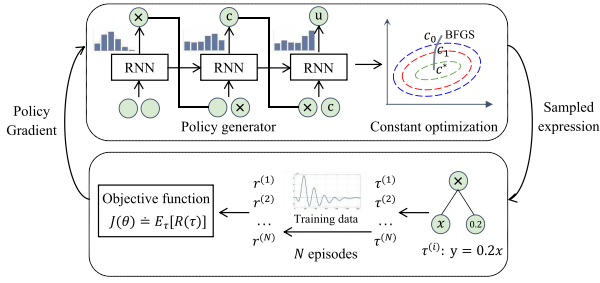


Fig. 1. Framework of the DSR model. An RNN generates sample probabilities, which are used to construct an expression tree. The predictive errors of the generated expressions serve as a reward signal to further train the RNN, improving expression generation over time.

gradient is adopted in the objective function to promote the generation of optimal mathematical expressions as follows:

$$J_{risk}(\theta) \doteq \mathbb{E}_{\tau \sim p(\tau|\theta)}[R(\tau)|R(\tau) \geq R_\varepsilon(\tau)]$$

$$R(\tau) = \frac{1}{\frac{1}{\sigma_y} \sqrt{\frac{1}{T} \sum_{t=1}^T (y_t - f(x_t))^2}} \quad (1)$$

where $R(\tau)$ is the reward, $R_\varepsilon(\tau)$ is the $(1 - \varepsilon)$ -quantile of the rewards, and σ_y is the standard deviation of the observations.

Other key components of the DSR design include constant optimization, constraints for avoiding invalid formulas, etc. (c.f. [16] for further details).

B. Threat Model

PLCs control the physical processes in ICSs. A PLC contains control logic and firmware that drives its operation. Configuration and updates to the PLC code are performed via a workstation PC equipped with PLC management software like TwinCAT 3, using a dedicated interface.

Similar to previous work on PLC security (e.g., [8]), we assume attackers can manipulate the PLC's control logic or modify its firmware to tamper with the PLC's output values. Remote attackers can gain access to a compromised workstation and use that to modify the control logic [19]. Other attack vectors, such as those at the firmware level, can be executed through direct hardware access, such as SD cards or the Joint Test Action Group (JTAG) interface [8], or through supply chain attacks. These advanced attacks can alter the output from PLCs in a more stealthy manner without leaving a visible trace on the PLC code.

We assume that the sensor readings are not modified in this work. Various false data detection and sensor reading protection methods have been extensively studied [20], and SRLR can complement these methods by enhancing control logic protection. Hence, in our threat model, SRLR is used to detect deviation of PLC's behavior, regardless of whether such deviation arises due to changes in application logic or modification in underlying firmware, and regardless of whether deviation happens persistently or occasionally.

III. SRLR: SYMBOLIC REGRESSION FOR PLC LOGIC RECOVERY & ATTACK DETECTION

We will present the design of SRLR in this section. After a brief introduction of DSR, we will describe the key techniques

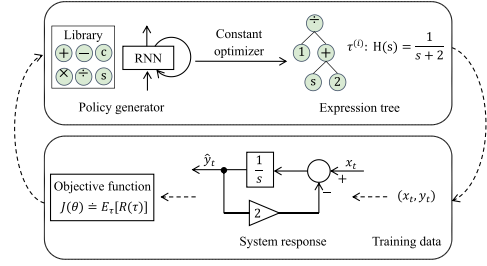


Fig. 2. Framework for single-mode process identification in the s -domain using the DSR model. The generated expressions represent transfer functions, and system responses are simulated to calculate the reward.

we incorporate into SRLR to make DSR work more effectively for PLC logic recovery, and consequently generate explainable attack detection rules.

A. Design 1: Using the Right Domain

1) *Single-Mode Logic Recovery*: The DSR method can be readily applied for single-mode logic recovery through the collection of input and output data from dynamic systems. However, oftentimes, specifically in ICSs, the system's time domain exhibits considerable complexity. As such, it becomes challenging for DSR to accurately generate the correct expressions, particularly for functions containing integral or derivative expressions.

Considering the Linear Time-invariant System (LTI), which can be described as an ordinary differential equation:

$$\sum_{p=1}^P a_p \frac{d^p}{dt^p} y_t = \sum_{q=1}^Q b_q \frac{d^q}{dt^q} x_t \quad (2)$$

where a_p and b_q are constant coefficients, and P, Q represent derivative orders related to inputs and outputs, respectively.

Time-domain systems often involve complex integral and derivative equations in ICSs, posing a challenge for DSR method to generate complex expressions. Nevertheless, these integral and derivative formulations can be readily transformed into simpler expressions in the *frequency domain or the s domain*. Applying Laplace transform to both sides of Equation (2), results in the following expression:

$$Y(s) \sum_{p=1}^P a_p s^p = X(s) \sum_{q=1}^Q b_q s^q \quad (3)$$

where $s = \sigma + iw$ is the complex s -domain parameter, σ, w are real constants. Thus, LTI can be expressed in a much simpler and clear fashion as the transfer function in s domain:

$$H(s) = \frac{Y(s)}{X(s)} = \frac{b_0 + b_1 s + \dots + b_Q s^Q}{a_0 + a_1 s + \dots + a_P s^P} \quad (4)$$

2) *Model Construction and Training*: We adopt the DSR model as the foundation for directly inferring the system's transfer function from observational data, without relying on expert knowledge. A schematic illustration of the general framework for identifying control systems is depicted in Figure 2. The search library comprises not only basic mathematical symbols, such as $\{+, -, \times, \div, \text{constant}\}$, but also contains the complex parameter s . Unlike the logic recovery

in the time domain, the input variables (e.g. x_1, x_2) are not a part of the library. This is because our goal is to predict the actual expression of the transfer function in the s domain.

In our approach, an RNN acts as the policy to sequentially generate the predictive transfer function. To evaluate system behavior during training, we compute the corresponding time-domain system response. This involves two main steps. First, the transfer function is converted into a state-space representation using the controllable canonical form. Next, a numerical integration technique, such as the Euler method, is applied to produce the system's output in the time domain. This entire computation process can be efficiently implemented using standard control libraries [21].

Subsequently, the prediction errors are utilized to calculate the rewards, we get:

$$R(\tau) = \frac{1}{\frac{1}{\sigma_y} \sqrt{\frac{1}{T} \sum_{t=1}^T (y_t - \hat{f}(x_t))^2}} \quad (5)$$

where $y_t, \hat{f}(x_t)$ are the ground truth and prediction outputs respectively.

SRLR can operate in either the time domain or the s domain. The choice between these domains can be guided by slight domain knowledge about the system's behavior. In the time domain, it models the system's dynamics directly, while in the s domain, it generates the corresponding transfer function. Despite these differences, both approaches use the same data representation: time-domain input data are fed into the simulated system to produce time-domain outputs. The prediction error between the simulated and actual outputs is then used to compute a reward signal, which guides model training through a risk-aware policy gradient method.

B. Design 2: Use of Continuity Constraint

1) *Multi-Mode Logic Recovery*: In this section, we introduce the details of multi-mode logic recovery. Consider a controller comprised of K modes. For example, the notation $s_t = k$ indicates that the mode k is operational in the time step t . We can describe the dynamics of the k -th mode through the equation $y_t = f_k(x_t, s_t = k; \zeta_k)$, and the multi-mode control logic can be formulated as follows:

$$y_t = f(x_t, s_t; \zeta) = \begin{cases} f_1(x_t, s_t = 1; \zeta_1), (x_t, y_t) \in \mathcal{D}_1 \\ f_2(x_t, s_t = 2; \zeta_2), (x_t, y_t) \in \mathcal{D}_2 \\ \dots \\ f_K(x_t, s_t = K; \zeta_K), (x_t, y_t) \in \mathcal{D}_K \end{cases} \quad (6)$$

where the function $f(\zeta)$ represents the equation governing the mode and ζ is the parameters, and the dynamics $f(\zeta)$ can be linear or nonlinear. $\mathcal{D}_k = \{(x_t, y_t) | s_t = k\}$ denotes the input/output set of the k -th mode.

The mode equation $f_k(x_t, s_t = k; \zeta_k)$ and the mode dataset $\mathcal{D}_k$ are not accessible apriori. To correctly determine the logic of such a multi-mode system, we need to identify the equation $f(\zeta_k)$ of each mode, the sub-dataset $\mathcal{D}_k$ of each mode, and the total number K of modes.

The multi-mode process identification problem can be formulated as two optimization tasks. The first task involves

(correctly) partitioning the dataset into sub-datasets corresponding to each operating mode, while the second employs DSR on that sub-dataset to identify the accurate equation governing that mode. However, solving the joint optimization problem presents a non-trivial challenge. We introduce a binary vector $\gamma_k \in \mathbb{R}^T$ as the membership value, where the membership $\gamma_{k,t}$ indicates whether the input-output pair (x_t, y_t) belongs to k -th mode. If true, $\gamma_{k,t} = 1$; otherwise, $\gamma_{k,t} = 0$. The joint objective function is formulated as follows:

$$\begin{aligned} \min_{\gamma, \zeta} \quad & \sum_{t=1}^T \sum_{k=1}^K \gamma_{k,t} l(y_t, \hat{y}_t; \zeta_k) + \phi L(\hat{y}_t) + g(\gamma_{k,t}; \lambda) \\ \text{s.t.} \quad & (x_t, y_t) \in \mathcal{D} \\ & \gamma_k \in \{0, 1\}^{|\mathcal{D}|} \\ & \hat{y}_t = f_k(x_t, s = k; \zeta_k) \end{aligned} \quad (7)$$

where $\hat{y}_t$ represents the predicted output at time step t . $\mathcal{D} = \bigcup_{k=1}^K \mathcal{D}_k$ and $|\mathcal{D}|$ is the total number of input-output pairs. The first term $l(y_t, \hat{y}_t; \zeta_k)$ is the prediction error for the equation of k -th mode (e.g. square error). The second term $L(\hat{y}_t)$ is the regularization for the prediction model with hyperparameter ϕ . The third term $g(\gamma_{k,t}; \lambda)$ is the regularization assigning the input-output pairs to a specific mode in training.

This is a general framework that can be utilized for multi-mode process identification tasks by specifying a regressor to predict system's output and defining a regularizer for partitioning the sub-dataset. We employ DSR as the regressor, and choose a hard regularizer that is widely adopted in previous studies [22]. The regularizer is formulated as $g(\gamma; \lambda) = -\lambda\gamma$.

We employ the Alternative Optimization Strategy (AOS) [22] to address the mixed-integer programming problem. AOS is an alternatively iterative method where we first fix the γ value and update the regressor model. Then, we fix the regressor parameters and update the γ value. The objective function for optimizing the regularizer is as follows:

$$\begin{aligned} \min_{\gamma} \quad & \sum_{t=1}^T \sum_{k=1}^K \gamma_{k,t} l_{k,t} - \lambda \gamma_{k,t} \\ \text{s.t.} \quad & \gamma_k \in \{0, 1\}^{|\mathcal{D}|} \end{aligned} \quad (8)$$

where $l_{k,t}$ is the abbreviation of $l(y_t, \hat{y}_t)$ for convenience. When the regressor parameters are fixed, the loss of prediction is a scalar. Thus, the optimal $\gamma_{k,t}$ can be obtained as follows:

$$\gamma_{k,t}^*(\lambda, l_{k,t}) = \begin{cases} 0 & \text{if } l_{k,t} \leq \lambda \\ 1 & \text{if } l_{k,t} > \lambda \end{cases} \quad (9)$$

2) *Refined Optimization Approach*: If the prediction loss $l_{k,t}$ is less than the pre-defined threshold λ , then the input-output pair (x_t, y_t) is assigned to mode k ($\gamma_{k,t} = 1$). Conversely, if the loss surpasses the threshold, it indicates that (x_t, y_t) is not generated by k -th mode ($\gamma_{k,t} = 0$). Given that fewer points in a multi-mode process provide limited information about the subsystem, we make the following basic hypothesis for constructing the identification method. This hypothesis holds for most ICS as mode switches are often non-transient in nature.

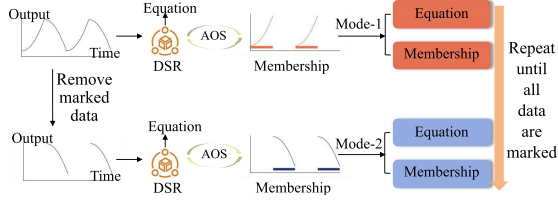


Fig. 3. Example of multi-mode logic recovery. The DSR model is iteratively optimized alongside a binary variable. Identified mode points are progressively removed, and the process continues until all points are assigned to a mode.

Assumption 1: At least w input-output pairs are generated during each mode activation (i.e., a mode switch can only occur after w time slots), where w is some small constant.

Considering that the data generation process in dynamic systems follows a temporal sequence, we leverage this smooth information and incorporate a regularization term to prompt the algorithm to commence the identification process with a greater relaxation for the first w data points, i.e., a higher value of λ . When we are identifying the k -th mode, the objective function can be expressed as follows:

$$\min_{\gamma_k} \sum_{t=1}^T \gamma_{k,t} l_{k,t} - \sum_{t=1}^w \lambda_1 \gamma_{k,t} - \sum_{t=w+1}^T \lambda_2 \gamma_{k,t} \quad (10)$$

s.t. $\gamma_k \in \{0, 1\}^{|D|}$

where λ_1 is a larger threshold that relaxes the condition for the initial points to be divided to mode k , and λ_2 is a smaller value that limits the predictions with minor errors. In our experiments, the parameters λ_1 and λ_2 are empirically set to 1 and 0.001, respectively. When the γ_k is fixed, we proceed to train the DSR model using the selected training data specified by γ_k . The objective function is defined as follows:

$$\min_{\zeta_k} \sum_{t=1}^T \gamma_{k,t} l(y_t, \hat{y}_t; \zeta_k) + \phi L(\hat{y}_t) \quad (11)$$

s.t. $(x_t, y_t) \in D$
 $\hat{y}_t = f_k(x_t, s = k; \zeta_k)$

We can employ risk policy gradient to optimize the DSR problem, and the gradient is computed as follows when identifying mode k :

$$\nabla_{\theta} J_{risk}(\zeta_k; \varepsilon) \approx \frac{1}{\varepsilon T} \sum_{t=1}^T \mathbb{I} \cdot [R(\tau^{(t)}) - \tilde{R}_{\varepsilon}(\zeta_k)] \nabla_{\zeta_k} \log p(\tau^{(t)} | \zeta_k) \quad (12)$$

where $\tilde{R}_{\varepsilon}(\zeta_k)$ is the $(1 - \varepsilon)$ quantile of the rewards in a training batch. $\mathbb{I}$ is an indicator variable, where if $R(\tau^{(t)}) \geq \tilde{R}_{\varepsilon}(\zeta_k)$, then $\mathbb{I} = 1$; otherwise, $\mathbb{I} = 0$.

As shown in Figure 3, the proposed method iteratively identifies each mode and removes the index once the sub-dataset is successfully partitioned.

C. Design 3: Outlier-Aware Training

While DSR is optimized in the previous section to handle the right domain and multiple modes for controller logic recovery, it may still be susceptible to outliers in the training data. These outliers can arise from environmental interference,

noise during data collection, or internal system vibrations. Such sensitivity can result in DSR generating overly complex expressions, leading to overfitting or even failure to recover control logic. To mitigate this issue, we proposed a data dropout approach to address potential outliers.

Rewards are calculated upon the generation of an expression by DSR, and these rewards are then utilized in the risk-seeking policy gradient mechanism to update the parameters of DSR. Throughout this process, all training data, including outliers, contribute to the reward calculation. Typically, the evolutionary models attempt to initially fit easier data in the early iterations and then tackle more challenging data (such as outliers) in subsequent iterations. Consequently, the normal data are likely to be fitted by generated candidate expressions, resulting in higher rewards for these data points and lower rewards for outliers. Drawing from this observation, we implement a data dropout approach where we remove the data representing the lowest α percent in terms of reward for a chosen constant α , as these are deemed more likely to be outliers.

D. Design 4: Complexity Regularization

In general, high-order systems featuring a more complex transfer function can emulate the response of lower-order systems when provided with input data. Consequently, it is essential to impose additional constraints on DSR to ensure the generation of expressions with suitable complexity levels.

We quantify complexity based on the number of mathematical symbols [16]. Specifically, the symbols $[\div, \times]$ exhibit higher complexity compared to $[+, -]$ as detailed in Table II, and the complexities are summed as the final complexity of the expression. We utilize the Akaike Information Criterion (AIC) [23] as a regularization term to encourage our model to generate less complex expressions. AIC is an indicator of predictive error derived from information theory, serving as a criterion for model selection in regression and balancing precision with simplicity. Its formulation is as follows:

$$AIC = (2m - \ln(\hat{L}))/n \quad (13)$$

where m is the number of model parameters, $\hat{L}$ is the predictive likelihood and n is the total number of data.

The preferred candidate model typically exhibits a lower AIC score, indicating either a lower complexity or a higher likelihood. Assuming the error residuals follow an independent identical normal distribution, the maximum value of a model's log-likelihood can be formulated by the residual. As our calculation of the complexity of generated expressions is based on the number of symbols, the complexity term tends to be small. To enhance the impact of expression complexity, we

TABLE II
THE MEASUREMENT OF SYMBOL COMPLEXITY

Complexity	1	2	3	4
Symbol	$+, -, \times,$ constant, variable	$\div$	$\sin, \cos$	$\log, \exp$

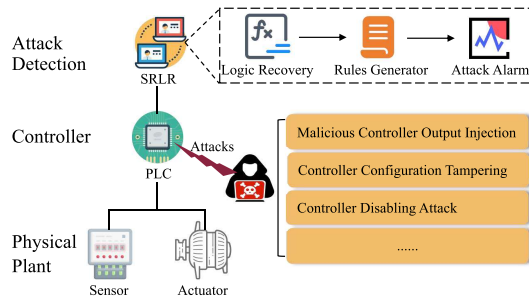


Fig. 4. Attack detection framework based on SRLR. The model learns normal PLC logic from sensor and actuator data, then generates rules to monitor and ensure PLC security.

introduce an additional complexity term into the AIC formula. The final modified AIC is calculated as follows:

$$\ln(\hat{L}) = -\frac{n}{2} \cdot \ln(2\pi) - \frac{n}{2} \cdot \ln\left(\frac{sse}{n}\right) - \frac{n}{2}$$

$$AIC = (2m - \ln(\hat{L}))/n + m \quad (14)$$

where sse is the residual error.

The negative AIC is a regularization term within the reward function that encourages the model to generate expressions that strike a balance between complexity and precision.

E. Using SRLR for Attack Detection

We now present how our proposed SRLR can be applied to detect different attacks in ICSs. The overall framework is depicted in Figure 4, where the ICSs comprise both the physical process and the controller module. Sensors measure the states of the physical system, such as water level, flow rate, etc., and these measurements are fed into PLCs, which analyze the system states and emit control commands to actuators. The actuators then execute these commands to control the physical system. However, PLCs are susceptible to vulnerabilities and may be targeted by attackers, leading to the transmission of incorrect or even harmful commands.

Notice that, once we have correctly identified the normal system process (in either single-mode or multi-mode), we can leverage that to check the behavior of the system. If there is a deviation from the expected behavior, the system can be considered as behaving anomalously and “under-attack”. These predictions are compared with the actual output from the actuators, with anomaly score calculated as:

$$e_t = \frac{1}{T} \sum_{t=1}^T |y_t - \hat{y}_t| \quad (15)$$

where y_t represents the observed PLCs output and $\hat{y}_t$ denotes the prediction of the identified system.

The output commands may exhibit some fluctuations, especially at the state-switching time points (during change in the operating mode), resulting in spikes in the anomaly scores. To prevent these points from being erroneously classified as an attack, we employ the Exponentially Weighted Moving Average (EWMA) smoothing algorithm to smooth the anomaly scores. Points are only flagged as attacks when the smoothed

anomaly score exceeds a predefined threshold determined through a non-parametric threshold [24].

The SRLR model can be integrated into monitoring systems within ICSs to deliver accurate numerical results that support timely and efficient alarms. In addition to its predictive capabilities, SRLR generates human-understandable rules, enabling operators to trace potential threats by analyzing these rules. Furthermore, it identifies compromised variables based on the generated rules, which is the critical information that helps security teams investigate and determine the root cause of incidents.

IV. EXPERIMENTS ON LOGIC RECOVERY

In this section, we conduct experiments to validate the performance of SRLR in terms of correctly identifying the operating mode of an ICS system.

A. Metrics

We evaluate the correctness of the recovered logic from two key perspectives: fidelity and explainability.

Fidelity refers to how accurately the recovered logic reproduces the true behavior of the system. To quantify this, we use the Best Fit Ratio (BFR) as defined in [25]. The BFR is calculated as follows:

$$\text{BFR} = \max \left\{ 0, 1 - \frac{\|y - \hat{y}\|_2}{\|y - \bar{y}\|_2} \right\} \quad (16)$$

where $y = [y(1), \dots, y(T)]$, $\hat{y} = [\hat{y}(1), \dots, \hat{y}(T)]$ are the ground truth and predicted system outputs, respectively. The BFR ranges from 0 to 100%, with higher values indicating closer alignment with the true system dynamics. This metric also serves as an indicator of model precision: if the recovered logic consistently achieves high BFR scores across test data, it suggests strong generalization and reliable predictive accuracy.

Explainability evaluates how easily a human can interpret the recovered expressions and understand the relationships among variables. First, we assess the overall symbolic complexity of each mode identified by SRLR, as shown in Table II. This complexity measure considers both the number and diversity of symbols used. Lower complexity generally implies greater explainability. Second, we examine symbolic equivalence, i.e. whether the recovered expression matches the known system dynamics. When the true dynamics are available, we can use intuitive visual inspection or formal tools such as SymPy¹ to determine equivalence.

This focus on explainability is a key advantage of SRLR. Unlike many machine learning-based approaches that yield only black-box numerical predictions, SRLR produces explicit symbolic expressions that not only approximate system behavior but also provide insight into the underlying control logic and variable relationships.

B. Single-Mode Logic Recovery

For the first evaluation, we tasked SRLR to recover the EWMA equations in the time domain. EWMA is a popular

¹docs.sympy.org.

TABLE III
THE RECOVERY RESULTS OF SRLR FOR THE EWMA EQUATION. GROUND TRUTH: $y(t) = 0.8x(t) + 0.16x(t-1) + 0.032x(t-2)$

Dataset	AIC	Outlier-aware	Identified mode	Complexity	BFR
Raw data	×	×	$y(t) = 0.2307x(t-1) + x(t)/(0.2585x(t-1)/(x(t-1) + x(t-2)) + 1.1842)$	19	0.9966
	×	×	$y(t) = 0.9064x(t-2)x(t)/(0.873x(t-2) - 0.1269x(t-1) + 0.1673x(t) + 0.1178)$	22	0.9963
	✓	✓	$y(t) = 0.7958x(t) + 0.1653x(t-1) + 0.0308x(t-2)$	15	0.9959
	✓	✓	$y(t) = 0.7917x(t) + 0.1706x(t-1) + 0.0296x(t-2)$	17	0.9957
Contaminated data	×	×	$y(t) = 0.0029x(t-1)(x(t-2)(-x(t-2) + x(t) + 641.6587) - 303.4484x(t-2) + x(t) - 8.2933)/x(t-2)$	29	0.8293
	×	✓	$y(t) = 0.7999x(t) + 0.2x(t-1)(0.0005x(t-2) - 0.0005x(t-1) + 0.96) - 0.0007$	24	0.9959
	✓	×	$y(t) = 0.9919x(t) - 0.2275x(t-1) + 0.2275x(t-2)$	10	0.9897
	✓	✓	$y(t) = 0.7985x(t) + 0.1628x(t-1) + 0.0305x(t-2)$	15	0.9945

and prevalent method employed on PLCs to preprocess data for smoothing in ICSs [26]. EWMA smooths time series data through an exponential window, expressed as follows:

$$y(t) = \alpha x(t) + (1 - \alpha)y(t-1) \\ \approx \alpha x(t) + \alpha(1 - \alpha)x(t-1) + \alpha(1 - \alpha)^2x(t-2) \quad (17)$$

where α is the smoothing factor, $x(t)$ is the input data at t -th time step and $y(t)$ is the output after EWMA process. Notice from the definition of EWMA – earlier inputs have progressively less influence on the output than recent ones. For our experiments, we truncate the input with a finite window size 2 and set the α to 0.8 to keep the resulting equation simple. However, our method also works when considering a larger number of inputs. Additionally, to simulate a contamination situation, we randomly introduce outliers to the raw data points as: $\tilde{x}(t) = \bar{x} \cdot \epsilon + x(t)$, where $\bar{x}$ is the local mean, and $\epsilon \sim \mathcal{N}(-0.1, 0.1)$.

The dataset utilized in this study is sourced from the real-world SWaT testbed [17]. The identification results are presented in Table III, revealing that the model can generate expressions with high BFR when outliers are not injected. However, without AIC regularization, the model tends to generate complex expressions that deviate significantly from the ground truth. The AIC regularization helps constrain the model to generate the correct expression with a lower complexity. In contaminated data, i.e., data with injected outliers AIC regularization alone results in incorrect expressions. However, when equipped with both the AIC regularization and outlier-aware method, the model can generate correct expressions with lower complexity and a high BFR. Experimental results confirm the effectiveness of our proposed AIC regularization and outlier-aware method, especially in cases where the training data contains outliers.

C. Multi-Mode Logic Recovery

Building upon the findings from single-mode logic recovery, we evaluate the performance of SRLR on identifying hybrid ICS systems having multiple operating modes.

1) *Dataset*: We consider four classic hybrid dynamic systems as in [12]. These hybrid dynamic systems include hysteresis relay, continuous hysteresis loop, phototaxic robot, and nonlinear system. The hysteresis relay and continuous hysteresis loop represent discrete and continuous switch systems, respectively. The phototaxic robot system comprises three modes to track the movement of a light-interacting robot. The nonlinear system comprises three modes with nonlinear equations. For further datasets details, please refer to [12].

We also evaluate the efficacy of SRLR across varying levels of noise in the data, comparing its performance against several baseline models. To this end, we consider an addition of zero-mean Gaussian noise to the input data, with noise levels ranging from 0% to 10% [12]. The noise level is proportional to the standard deviation of the raw data, denoted as $N_p = \sigma_{noise}/\sigma_y$, where σ_{noise} represents the standard deviation of the noisy data and σ_y is the standard deviation of the noiseless data. We use the noisy data as the training set and the noiseless data as the testing set following the evaluation in [12] to ensure a fair comparison.

2) *Baselines*: We evaluate the performance of SRLR against six baseline methods. The first, Cluster SR [12], is a symbolic regression technique that combines Generic Programming (GP) with the expectation-maximization algorithm for clustering and membership estimation. To benchmark SRLR against SOTA symbolic regression tools, we also create three variants: Cluster DSR, Cluster PySR, and Cluster Operon. These versions replace the GP component in Cluster SR with more advanced symbolic regression algorithms, such as DSR, PySR [27], and Operon [28], respectively. For broader comparison, we also include two black-box deep learning models: Long Short-term Memory Network (LSTM) and a Deep Cascaded Neural Network (CascadedNN) [9].

3) *Comparison Results*: As shown in Figure 6, SRLR consistently outperforms all baseline methods in terms of average BFR. The black-box models, LSTM and CascadedNN, demonstrate competitive results on the discrete and continuous hysteresis datasets, where their performance closely matches SRLR. However, their effectiveness declines on more complex tasks, such as modeling nonlinear systems. In contrast, SRLR not only maintains high predictive accuracy but also provides interpretable equations that reveal the underlying system dynamics.

The performance of the cluster-based symbolic regression models varies across datasets. For example, Cluster SR achieves better results on the phototaxic robot dataset but struggles with the continuous hysteresis system, leading to a significantly lower BFR, as shown in Table IV. This inconsistency can be attributed to the shared membership estimation method across all cluster-based models, which fails to account for the temporal continuity present in system-generated data. As a result, these models underperform compared to SRLR, particularly on tasks that require capturing complex or sequential patterns.

Furthermore, we conducted a comparative analysis of the mode indexes identified by SRLR (ours) and Cluster SR. Both approaches are trained using noisy data, and the resulting

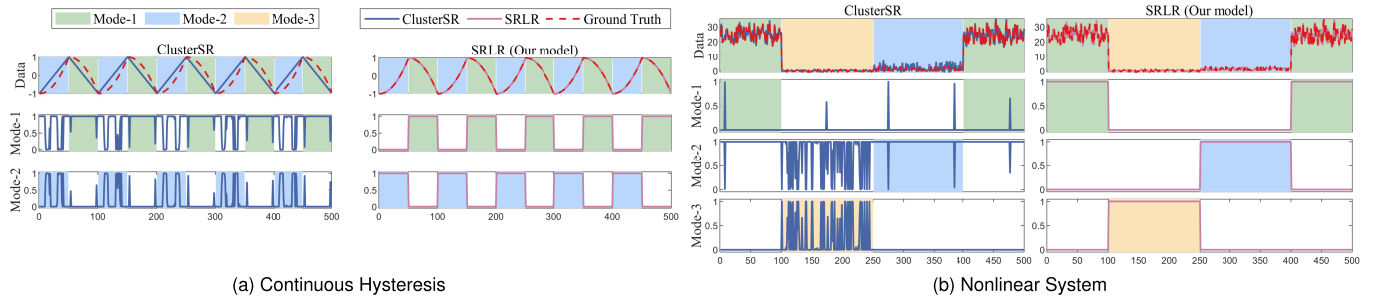


Fig. 5. Comparison of mode identification results between Cluster SR and the proposed SRLR method. The first row shows simulated data from both methods alongside ground truth. Subsequent rows present the identified mode indices for each method.

TABLE IV
THE IDENTIFICATION RESULTS OF SRLR AND BASELINES IN MULTI-MODE LOGIC RECOVERY. (NOISE LEVEL $N_p = 0.02$)

Dataset	Mode	The ground truth mode	SRLR	Cluster SR	Cluster DSR
Hysteresis Relay	1	$y = 1$	$y = 1$	$y = 1$	$y = 1$
	2	$y = -1$	$y = -1$	$y = -1$	$y = -1$
Continuous Hysteresis	1	$y = 0.5x^2 + x - 0.5$	$y = 0.4988x^2 + 0.9993x - 0.4994$	$y = 0.013 + x$	$y = 0.4988x^2 + 0.9993x - 0.4994$
	2	$y = -0.5x^2 + x + 0.5$	$y = -0.4987x^2 + 0.9998x + 0.4996$	$y = x$	$y = -0.4987x^2 + 0.9997x + 0.4995$
Phototaxic Robot	1	$y = x_2 - x_1$	$y = x_2 - x_1$	$y = x_2 - x_1$	$y = x_2 - 1.0009x_1$
	2	$y = 1/(x_1 - x_2)$	$y = 0.9647/(1.0332x_1 - 1.0332x_2)$	$y = 0.969/(x_1 - x_2)$	$y = 0.083x_1 - 0.083x_2 - 0.0873$
	3	$y = 0$	$y = 0$	$y = -0.005$	$y = 0$
Non-linear System	1	$y = x_1x_2$	$y = x_1x_2$	$y = x_1x_2$	$y = 1.0827x_1x_2 - 0.4131x_1$
	2	$y = 6x_1/(6 + x_2)$	$y = 5.9206x_1/(5.9153 + x_2)$	$y = x_1x_2 - (x_1 - 5.169)x_1/(-6.699)$	$y = 0.4774x_1^2 + 0.4774x_1x_2$
	3	$y = (x_1 + x_2)/(x_1 - x_2)$	$y = (x_1 + x_2)/(x_1 - x_2)$	$y = (x_1 + x_2)/(x_1 - x_2)$	$y = 1.2327x_1x_2 - 1.2797x_1$

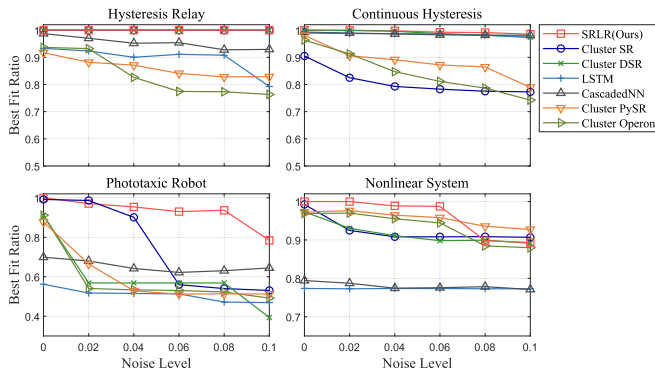


Fig. 6. The comparative BFR across different noise levels.

index identification results are illustrated in Figure 5. A key difference we observe is that Cluster SR does not account for the temporal dynamics of systems and therefore overlooks the local continuity of the point distribution within each mode.

As shown in Figure 5, Cluster SR yields a sparse membership distribution in the continuous hysteresis system, despite most values being correct. In the case of the nonlinear system, Cluster SR even misclassifies the membership due to the presence of noisy training data. In contrast, as SRLR leverages temporal information and employs a larger λ_1 (see Equation 10), along with an effective fine-tuning mechanism, it helps the correct identification of the initial points of each mode, providing the algorithm with essential prior information to guide its decisions. This leads to the correct identification of initial segments, and inference of the correct mathematical structure which results in overall better performance.

TABLE V
THE PARAMETERS AND RESULTS OF THE DISTRIBUTION GRID

System	Gain	Time Constant	Total Number	Average BFR
Amplifier	(10, 400)	(0.02, 0.1)	79	0.9999
Exciter	(0.7, 1)	(0.02, 1)	79	0.9999
Generator	(0.7, 1)	(1, 2)	79	0.9999
Sensor	(0.7, 1)	(0.01, 0.06)	79	0.9999
Controller	Automatically tuned		79	0.9969

D. Logic Recovery in Distribution Grid

Next we evaluate SRLR on a distribution grid with hundreds of voltage regulators are deployed and updated independently to stabilize the local voltages delivered to users. To simulate this distributed network, we randomly generated the gains and time constants for each subsystem within the normal range, as referenced in the literature [29]. The corresponding parameters of Proportional-Integral-Derivative (PID) controllers were automatically tuned using MATLAB PID Tuner App.

As illustrated in Table V, we simulate 40 distributed voltage regulator systems, each consisting of four physical subsystems and one PID controller. The recovery performance of SRLR in physical subsystems (amplifier, exciter, generator, and sensor) achieved an average BFR 0.9999, demonstrating SRLR's stability under diverse conditions. Additionally, the average BFR for the controllers was 0.9969, indicating SRLR's effectiveness in identifying complex logic (e.g. second-order mathematical expression in PID controllers) with a high degree of accuracy. In total, 395 subsystems were identified across 79 distributed systems, and the high success rate highlights SRLR's robustness and suitability for deployment in large-scale, distributed real-world systems.

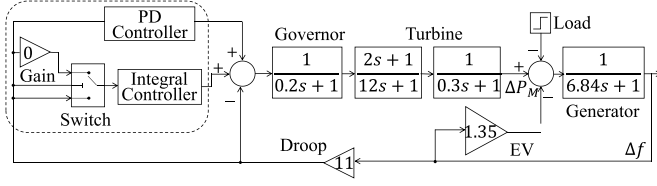


Fig. 7. Framework of the simulated LFC system. The system uses an integral-separated PID controller, with logic that switches based on feedback signals.

V. EXPERIMENTS ON ATTACK DETECTION

In this section, we evaluate the performance of SRLR to detect controller logic attacks in different ICS environments, including a power grid frequency control system [30] and the SWaT testbed [17].

A. Metrics and Baselines

1) *Metrics*: We evaluate detection performance using widely-used metrics: F1 score, precision, and recall. Following established practice in prior work, we enumerate all possible threshold values and report the highest achieved F1 score, denoted as F1, Precision, and Recall. Additionally, we adopt the adjust-point evaluation strategy [4], [15], [31], which considers an entire attack segment correctly detected if at least one alarm is triggered within its duration. Metrics calculated using this approach are reported as F1, Precision, and Recall. The average performance metrics along with their standard deviations are reported.

2) *Baselines*: We compare SRLR against several neural System Identification (SI) methods, including RNN [9], CascadedNN [9], and NSIBF [4] along with its variants NSIBF-RECON and NSIBF-PRED. RNN and CascadedNN directly predict system outputs using neural networks. NSIBF constructs a state-space model via neural networks and updates system states through Bayesian filtering. It detects anomalies based on a combination of reconstruction and prediction errors. In addition to these neural SI methods, we also evaluate SRLR against deep learning-based anomaly detection models. These include the encoder-decoder architecture EncDec-AD [32], the density-based approach DAGMM [33], the adversarially trained method USAD [15], and the probabilistic modeling framework OmniAnomaly [31].

B. Power Grid Frequency Control System

1) *System Description*: We simulated a power grid Load Frequency Control (LFC) system as the first ICS case study. The system parameters are aligned with prior research simulating a power system of Great Britain [30]. As depicted in Figure 7, the controller is modeled using an integral-separated PID controller. The integral component of the PID is crucial for eliminating static errors; however, in instances where the input errors are substantial, the integral controller accumulates these errors, potentially driving the system's actuators into a state of saturation – a phenomenon extensively studied and referred to as controller windup in the literature [34], [35], [36]. Integral-separated PID control is an effective strategy

TABLE VI
THE CONTROLLER IDENTIFICATION RESULTS

Mode	Ground truth	Prediction
Controller-1	$\frac{3.3s+2.1}{s+1}$	$\frac{3.874s+2.599}{1.182s+1.231}$
Controller-2	$\frac{3.3s^2+2.7s+0.6}{s^2+s}$	$\frac{3.027s^2+2.099s+0.4469}{s^2+0.6936s}$
Switch threshold	0.1	0.0979

to circumvent windup; it operates by switching the controller to Proportional-Derivative (PD) mode when the controller's input errors exceed a predefined threshold, and reverting to PID mode when inputs fall below this threshold. The training dataset consists of 180,000 simulated points, while the testing dataset includes 182,500 points, with a 39.6% anomaly rate generated through various attack scenarios described in the attack model section.

2) *Attack Model*: We launch three types of attacks on the LFC system, aimed at disturbing the frequency deviation and potentially causing the frequency control to fail. The first attack is *Malicious Controller Output Injection Attack*. Similar to previous methods of injecting malicious data [37], we assume these attacks occur randomly, governed by a nonlinear function, formulated as follows:

$$\begin{aligned} \bar{y}(t) &= \beta(t)g(y(t)) + (1 - \beta(t))y(t) \\ g(y) &= -\tanh(Gy) \end{aligned} \quad (18)$$

Here, G represents the constant for the upper bound, and $\beta \in \{0, 1\}$ is a Bernoulli variable with an expectation ranging from $[0, 0.3]$. $g(y)$ generates the malicious data with the restriction $\|g(y)\|^2 \leq \|Gy\|^2$.

The second attack is the *Controller Configuration Tampering Attack*, where the parameters of the PID controller are altered based on a random variable:

$$\begin{aligned} KP &= KP + \xi_1, KI = KI + \xi_2, KD = KD + \xi_3 \\ KP &\geq 0, KI \geq 0, KD \geq 0 \end{aligned} \quad (19)$$

where ξ_1, ξ_2, ξ_3 follow a Gaussian distribution $\mathcal{N}(0, 1)$. KP , KI , and KD represent the proportional, integral, and derivative parameters of a PID controller, respectively.

The third attack is *Controller Disabling Attack*. Specifically, $KP = 0, KI = 0, KD = 0$. While this attack shares similarities with tampering with the PID configurations, it differs in that the system's PID configuration returns to normal after a short period. This attack can be formulated as follows:

$$\begin{aligned} \text{PID config} &= \text{normal}, t \in [0, T_{off}) \\ \text{PID config} &= \text{attack}, t \in [T_{off}, T], \end{aligned} \quad (20)$$

where the controller outputs are divided into N periods, and each period lasts for T time slots. T_{off} is the starting time step for attacks in one period.

3) *Attack Detection*: We utilize the SRLR in multi-mode logic recovery setting to identify the switched controller logic (i.e., logic switch from PD control to PID control) utilizing just the controller inputs and outputs as training data. Subsequently, we apply the identified controllers' logic to the dataset and compute the errors, determining the time points where a mode switch occurs. Employing the Mahalanobis

TABLE VII
THE COMPARISON OF ATTACK DETECTION IN THE LOAD FREQUENCY CONTROL SYSTEM

	Model	F1	Precision	Recall	F1*	Precision*	Recall*
Malicious Controller Output Injection	CascadedNN	0.9916 ± 0.0126	0.9937 ± 0.011	0.9896 ± 0.0166	0.9939 ± 0.0096	1.0 ± 0.0	0.988 ± 0.0189
	RNN	0.9795 ± 0.0315	0.9901 ± 0.0222	0.9700 ± 0.0463	0.9822 ± 0.0273	0.9927 ± 0.0192	0.9723 ± 0.0381
	EncDec-AD	0.3357 ± 0.1028	0.2262 ± 0.0706	0.6690 ± 0.1966	0.3543 ± 0.0973	0.2430 ± 0.0735	0.6749 ± 0.1526
	DAGMM	0.2732 ± 0.1333	0.1646 ± 0.0889	1.0 ± 0.0	0.2826 ± 0.0152	0.1646 ± 0.1054	1.0 ± 0.0
	USAD	0.8521 ± 0.1802	0.7772 ± 0.2359	0.9991 ± 0.0003	0.8943 ± 0.1169	0.8323 ± 0.1768	0.9915 ± 0.0159
	OmniAnomaly	0.8121 ± 0.1011	0.6964 ± 0.1595	0.9963 ± 0.0063	0.9874 ± 0.0213	0.9961 ± 0.0094	0.9797 ± 0.0416
	NSIBF	0.8727 ± 0.0929	0.8683 ± 0.1776	0.8862 ± 0.0037	0.9391 ± 0.0860	0.9107 ± 0.1262	0.9717 ± 0.0398
	NSIBF-RECON	0.9245 ± 0.1009	0.8695 ± 0.1768	0.9981 ± 0.0012	0.9318 ± 0.0964	0.8940 ± 0.1498	0.9776 ± 0.0316
	NSIBF-PRED	0.9245 ± 0.1009	0.8695 ± 0.1768	0.9981 ± 0.0012	0.9383 ± 0.0872	0.9030 ± 0.1370	0.9803 ± 0.0278
	SRLR (Ours)	0.9980 ± 0.0013	1.0 ± 0.0	0.9961 ± 0.0027	0.9982 ± 0.0015	1.0 ± 0.0	0.9964 ± 0.0030
Controller Configuration Tampering Attack	CascadedNN	0.9736 ± 0.0571	0.9582 ± 0.0963	0.9951 ± 0.0141	0.9999 ± 0.0	1.0 ± 0.0	1.0 ± 0.0
	RNN	0.9286 ± 0.0841	0.9069 ± 0.1285	0.9662 ± 0.0655	0.9999 ± 0.0	1.0 ± 0.0	1.0 ± 0.0
	EncDec-AD	0.7721 ± 0.1402	0.6881 ± 0.2218	0.9544 ± 0.0914	0.9960 ± 0.0112	0.9922 ± 0.0214	1.0 ± 0.0
	DAGMM	0.7014 ± 0.0923	0.5485 ± 0.1074	0.9950 ± 0.0161	0.7087 ± 0.0915	0.5560 ± 0.1074	1.0 ± 0.0
	USAD	0.7555 ± 0.1274	0.6956 ± 0.2275	0.9201 ± 0.1359	0.9824 ± 0.0242	0.9664 ± 0.0456	1 ± 0.0
	OmniAnomaly	0.7058 ± 0.0887	0.5552 ± 0.1015	0.9872 ± 0.0319	0.9399 ± 0.0848	0.8964 ± 0.1279	1.0 ± 0.0
	NSIBF	0.751 ± 0.0621	0.6717 ± 0.0657	0.8519 ± 0.0541	0.9877 ± 0.0151	0.9759 ± 0.0295	1.0 ± 0.0
	NSIBF-RECON	0.7080 ± 0.0458	0.5491 ± 0.0551	0.9996 ± 0.0004	0.9744 ± 0.0258	0.9755 ± 0.0141	0.9735 ± 0.0374
	NSIBF-PRED	0.7082 ± 0.0462	0.5493 ± 0.0554	1.0 ± 0.0	0.9713 ± 0.0204	0.9695 ± 0.0036	0.9735 ± 0.0374
	SRLR (Ours)	0.9955 ± 0.0077	0.9981 ± 0.0043	0.9929 ± 0.0114	0.9999 ± 0.0	1.0 ± 0.0	1.0 ± 0.0
Controller Disabling Attack	CascadedNN	0.9727 ± 0.0392	0.9707 ± 0.0455	0.9757 ± 0.0433	0.9999 ± 0.0	1.0 ± 0.0	1.0 ± 0.0
	RNN	0.8857 ± 0.0823	0.8853 ± 0.0706	0.8899 ± 0.1079	0.9888 ± 0.0185	0.9801 ± 0.0318	0.9981 ± 0.0083
	EncDec-AD	0.5026 ± 0.0796	0.3946 ± 0.1230	0.8703 ± 0.2398	0.9139 ± 0.0454	0.8912 ± 0.0499	0.9444 ± 0.0860
	DAGMM	0.4824 ± 0.0790	0.3213 ± 0.0680	0.9991 ± 0.0020	0.4904 ± 0.0805	0.3284 ± 0.0696	1.0 ± 0.0
	USAD	0.5056 ± 0.0734	0.3778 ± 0.1307	0.9394 ± 0.1854	0.8285 ± 0.0817	0.7738 ± 0.1005	0.9122 ± 0.1207
	OmniAnomaly	0.7229 ± 0.1020	0.6004 ± 0.1238	0.9324 ± 0.0912	0.9864 ± 0.0276	0.9745 ± 0.0506	1.0 ± 0.0
	NSIBF	0.5839 ± 0.0206	0.5165 ± 0.0518	0.6907 ± 0.1476	0.8593 ± 0.0085	0.8483 ± 0.0066	0.8705 ± 0.0105
	NSIBF-RECON	0.5012 ± 0.0137	0.3344 ± 0.0122	1.0 ± 0.0	0.7578 ± 0.0151	0.6221 ± 0.0119	0.9691 ± 0.0204
	NSIBF-PRED	0.5009 ± 0.0133	0.3342 ± 0.0119	0.9994 ± 0.0007	0.7699 ± 0.0323	0.6379 ± 0.0456	0.9725 ± 0.0030
	SRLR (Ours)	0.9822 ± 0.0264	0.9975 ± 0.0045	0.9685 ± 0.047	0.9999 ± 0.0001	1.0 ± 0.0	0.9999 ± 0.0001

¹ F1*, Precision* and Recall* refer to the metrics calculated using the point-adjusted evaluation strategy.

distance-based filtering method [38], we then utilize the DSR model to determine the logic switch threshold. The training process requires 5.7 hours. The results presented in Table VI, demonstrate the accurate recovery of the switched controller logic and the corresponding switch results.

The results are presented in Table VII. It is evident from the table that our model consistently outperforms the baselines across all three types of attack detection scenarios. This superiority can be attributed to the accuracy with which our model recovers the switched controllers' logic and the associated switch conditions. Notably, the CascadedNN exhibits the second-best performance owing to its fully connected characteristics. SI based methods, including our proposed model, RNN, CascadedNN and NSIBF, demonstrate superior performance compared to autoencoder-based approaches and generative approaches.

In the case of malicious controller output injection, our model effectively detects nearly all instances of malicious data. This success stems from the model's ability to accurately identify between malicious and normal data. Conversely, autoencoder methods struggle to distinguish between normal and malicious data, as they primarily focus on capturing the distribution of training and testing data without explicitly considering controller logic. When the controller configuration is tampered with, particularly if the tampered configuration closely resembles the normal configuration, the performance of our model, RNN, and CascadedNN experiences a slight decrease. However, EncDec-AD and DAGMM methods exhibit improved performance in this scenario, as the

tampered configuration persists for an extended period, leading to a continuous presence of attacked points. In contrast, the controller disabling attack poses a stealthier threat, as it only lasts for a brief duration and may go undetected by system identification-based methods. Moreover, the relatively fewer attacked points in comparison to controller configuration tampering also contribute to decreased performance for NSIBF, EncDec-AD, DAGMM and USAD. OmniAnomaly demonstrates relatively stable performance across three types of attacks, due to its ability to model the dynamic characteristics of time series data. Additionally, the point-adjusted metrics yield higher values compared to standard metrics, as they evaluate continuous attack segments as single anomaly events. This approach increases the likelihood of detection, particularly in cases such as controller configuration tampering, where identifying any point within the segment is sufficient to flag the entire attack. In summary, leveraging the identified switch controllers' logic and associated switch conditions enables accurate simulation of system behavior, facilitating efficient detection of attacks.

C. Secure Water Treatment Testbed

The SWaT platform, developed by iTrust SUTD [17], represents a scaled-down emulation of real-world water treatment systems with six-stage processes. In the first 7 days, the SWaT system operated under normal conditions, followed by four days of operation under attack conditions. The dataset, collected from 51 sensors and actuators, comprises 495,000

training samples and 449,919 testing samples, with approximately 12.14% of the testing set containing attacked points. These attacks were initiated by tampering with sensor or actuator values, introducing perturbations into the system dynamics.

The SWaT dataset has been extensively utilized in the machine learning community [4], [5]. Prior research primarily focuses on detecting attacks and triggering alarms without providing a comprehensive explanation for the occurrence of these attacks to system operators. Moreover, these studies often treat all attacks as a single category, overlooking the diverse nature of attack types. We address these limitations by conducting an in-depth analysis of attack behaviors, categorizing them into three distinct types. Subsequently, we apply our mode process identification method to recover controller logic, enabling the generation of rules for attack detection. Leveraging these generated rules, we detect attacks and provide detailed explanations to system operators when alarms are triggered. This approach not only enhances the accuracy of attack detection but also facilitates a deeper understanding of the underlying causes behind these security breaches.

1) *Attack Categories*: We classify the attacks observed in the SWaT dataset into three distinct types: a) *Actuator Value Tampering Attack*: The actuators themselves are targeted, resulting in the manipulation of their values. b) *Actuator-Dependent Sensor Value Tampering Attack*: The attacker tampers with sensor values that share a consistent relationship with specific actuators. This relationship becomes disrupted upon tampering with the sensor values. c) *Actuator-Independent Sensor Value Tampering Attack*: The attacker targets sensor values that are not directly related to the control logic of the PLCs governing the actuators. For our analysis, SWaT dataset encompasses 36 consecutive attacks, with 19 attacks classified as actuator value tampering, 6 attacks categorized as actuator-dependent sensor value tampering, and 11 attacks falling under the actuator-independent sensor value tampering category. This classification allows for a comprehensive understanding of the diverse attack scenarios present within the SWaT dataset.

2) *Attack Detection*: We grouped the 51 variables into six categories corresponding to the distinct physical processes within the system. Each group encompasses both actuators and sensors that relate to its specific physical process. It is assumed that some of the actuators are controlled by the value of sensors and this can be expressed through a mathematical formulation. Upon examining the actuator data, it was observed that actuators operate in binary states, either “on” or “off”, with these states typically determined by time-domain control logic. To reflect this behavior, SRLR is operated within the time domain on the SWaT system. To further enhance modeling accuracy, we introduced a step function. This adjustment not only better captures the binary nature of actuator outputs but also reduces the SRLR search space. With this configuration, SRLR is capable of capturing nonlinear relationships.

```
def rule_1(P205_obs):
    x1=AIT202; x2=AIT203; x3=FIT201
    x4=AIT203_delay; x5=FIT201_delay
    P205_pred = step(x1**3*x3-x2*x5+x2-2*x4)
    error = abs(P205_pred-P205_obs)
    return error
def rule_2(MV302_obs):
    x1=DPIT301; x2=FIT301;
    x3=LIT301; x4=FIT301_delay
    MV302_pred = step(x1-0.0115*x3)*(x2+x3+x4)
    error = abs(MV302_pred-MV302_obs)
    return error
def rule_3(LIT301_obs):
    if LIT301_obs>1020 or LIT301_obs<780:
        error = 1
    else:
        error = 0
    return error
```

Listing 1. Examples of invariant rules on the training dataset.

The dynamic relationship between an actuator and its sensors is represented by the following equation:

$$y(t) = \text{Step}(f(x_1(t), x_1(t-\tau), x_2(t), x_2(t-\tau) \dots)) = \begin{cases} 1, f(x) \geq 0 \\ 0, f(x) < 0 \end{cases} \quad (21)$$

where $y(t)$ represents the actuator value at time step t , $x(t)$, and $x(t-\tau)$ denote the current and delayed sensor values within the same group, respectively. $\text{step}()$ represents the step function. The function $f(x)$ reveals the relationship between the sensors and the actuator, with the actuator activated if $f(x) \geq 0$, and deactivated otherwise.

Employing the proposed single-mode logic recovery approach, we endeavored to recover the truth logic of the function $f(x)$ by identifying all relationships that invariantly hold true within the training dataset. Training on the SWaT dataset takes approximately 4.08 hours. This process generated 15 definitive rules that remained valid across the training set. Subsequently, these rules were applied to simulate the corresponding actuator values within the testing set, facilitating the detection of anomalies indicative of potential attacks.

For the first type of attack, i.e., the Actuator Value Tampering Attack, detection becomes feasible if rules governing actuator value generation are correctly established by our proposed SRLR model. Leveraging the single-mode process identification method, we derive the formula governing the relationship between actuator P205 and sensors in process 2, subsequently yielding Rule 1. As depicted in List 1, rows 2 and 3 are sensor observations assignments, row 5 denotes the generated formula, and row 6 represents the error between predictive P205 and observed P205. As illustrated in Figure 8(a), predictive P205 consistently aligns with observations in the training data. However, in the test data, the actuator P205 is maliciously manipulated from “on” to “off” during the attack, resulting in a discrepancy between predictive and observed P205 values and triggering an alarm. Thus, this actuator tampering attack is effectively detected.

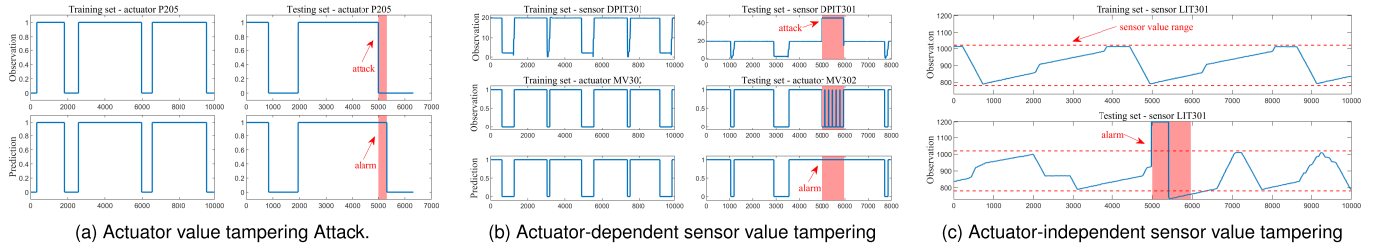


Fig. 8. Examples of attack detection results on the SWaT dataset with red areas highlighting the intervals where actual attacks occur. (a) The observed and predicted values for the pump P205. The left sub-figure shows the training set and the right sub-figure shows the results on the testing set. (b) observations of differential pressure DPIT301 and the observed and predicted behavior of valve MV302 on the training and testing sets. (c) Observations of level transmitter LIT301 on training and testing sets.

For the second type of attack, i.e., the Actuator-Dependent Sensor Value Tampering Attack, the sensor value is linked to the actuator. Although the sensor value does not directly control the output of the PLC, when the sensor value is tampered with, the system's status becomes abnormal, disrupting the previous invariant relationship between the sensor and actuator. Therefore, identifying the relationship between sensors and their corresponding actuators can detect this type of attack. As illustrated in List 1 rows 9 to 14, we establish the relationship between the actuator MV302 and the sensor value DPIT301. This equation consistently holds true in the training data, as depicted in Figure 8(b). In the testing data, the attacker tampers with the DPIT301 value, causing oscillations in the MV302 value. However, when simulating the logic in List 1 row 12, the MV302 is predicted to remain consistently open, triggering the alarm and thus, detecting the attack successfully.

For the third type of attack, i.e., the Actuator-Independent Sensor Value Tampering Attack, the sensor value neither controls the PLC's output nor has an invariant relationship with actuators. Therefore, we cannot employ the mode equation identification method to detect this type of attack. However, we can utilize the normal range observed in the training data to constrain the corresponding sensor values in the testing data. As illustrated in Figure 8(c), although the LIT301 participates in the calculation of MV302, we find that tampering with the LIT301 value does not affect the prediction for the MV302. Then, we analyze the range of LIT301 in the training data and generate a rule to limit the range of LIT301 in the testing data. As shown in List 1 rows 16 to 21, we observe that LIT301 in the training data falls within the range of [780,1020]. In the testing data, when the value of LIT301 is tampered to exceed the normal range, this attack can succeed, as depicted at the bottom of Figure 8(c). Thus, while this type of attack cannot be detected by the proposed model, we can still apply the range rule to detect out-of-range values.

We observe some (false) alarms when there is no attack. For instance, SRLR identifies a relationship between MV101 and sensor FIT101. However, there are instances of delayed responses / updates in the testing data, leading to triggering of alarms. In this case, SRLR recovers correct logic, but there is an additional task (beyond SRLR) to distinguish whether the violation of the rule is due to attack or some data anomaly.

3) *Comparison With Baselines:* We compare our results with various deep learning-based methods and neural

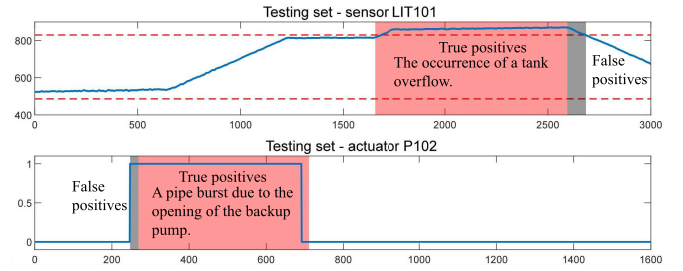


Fig. 9. The case study for false positives.

system identification models. As depicted in Table VIII, our model surpasses EncDec-AD, generation-based models USAD, DAGMM, OmniAnomaly, and neural system identification models CascadedNN, RNN and NSIBF. Specifically, although NSIBF (neural system identification method) can numerically identify systems, it fails to generate a clear expression, thereby lacking explanatory power for ICS operators. Once the true rules are generated by SRLR, attacks can be detected effectively, as most attacks involve control logic modifications, output value tampering, or sensor readings falling outside valid ranges. Consequently, the number of false negatives is significantly lower compared to deep learning methods, as evidenced by the recall metric in Table VIII. As previously discussed, the rare cases of misdetection are due to environmental sampling delays. As illustrated in Figure 9, these false positives occur either due to the persistence of the hazard (e.g., tank overflow) or due to the discrepancies of actuator-sensor relationship rules generated by SRLR. Although the data is labeled as normal in the gray area shown in Figure 9, these false alarms are justifiable in real-world systems. For the first false alarm, despite the valve being closed, the water level takes time to return to normal, leading to a delay that results in false positives (grey area). For the second alarm, our model detects an immediate violation of the actuator values as soon as the pump is activated, even though the pipe has not yet burst. This early detection leads to false positives. Furthermore, the false positive rate of SRLR is 0.0068. We already consider the noise and outliers, hence the number of false positives can be kept to a minimum. This results in a precision score consistently over 90%. The ICS-specific design of SRLR enables it to handle complex systems effectively, allowing it to generate rules for attack detection

TABLE VIII
THE COMPARISON OF ATTACK DETECTION IN THE SWAT SYSTEM

Model	F1	Precision	Recall	F1*	Precision*	Recall*
CascadedNN	0.6829 ± 0.0619	0.7473 ± 0.1647	0.6433 ± 0.0296	0.9010 ± 0.0062	0.9499 ± 0.0144	0.8573 ± 0.0170
RNN	0.6963 ± 0.0735	0.9386 ± 0.0457	0.5612 ± 0.1032	0.9048 ± 0.0184	0.9471 ± 0.0242	0.8668 ± 0.0340
EncDec-AD	0.6940 ± 0.0405	0.8272 ± 0.1517	0.6080 ± 0.0506	0.8762 ± 0.0185	0.9132 ± 0.0264	0.8423 ± 0.0183
DAGMM	0.2165 ± 0.0	0.1213 ± 0.0001	0.9999 ± 0.0	0.8533 ± 0.0097	0.8602 ± 0.0116	0.8464 ± 0.0079
USAD	0.6807 ± 0.0977	0.8240 ± 0.2933	0.6188 ± 0.0570	0.8534 ± 0.0172	0.9442 ± 0.0540	0.7836 ± 0.0703
OmniAnomaly	0.7540 ± 0.0018	0.9962 ± 0.0007	0.6066 ± 0.0026	0.7993 ± 0.0053	0.9993 ± 0.0009	0.6660 ± 0.0076
NSIBF	0.6983 ± 0.0053	0.7725 ± 0.0116	0.6371 ± 0.0017	0.8691 ± 0.0096	0.8273 ± 0.0156	0.9164 ± 0.0362
NSIBF-RECON	0.7603 ± 0.0037	0.9669 ± 0.0316	0.6270 ± 0.0182	0.8690 ± 0.0217	0.9481 ± 0.0103	0.8023 ± 0.0297
NSIBF-PRED	0.7417 ± 0.0037	0.9468 ± 0.0438	0.6106 ± 0.0216	0.8680 ± 0.0125	0.8660 ± 0.0337	0.8710 ± 0.0216
SRLR (Ours)	0.8329 ± 0.0007	0.9816 ± 0.0007	0.7233 ± 0.0015	0.9299 ± 0.0096	0.9353 ± 0.0197	0.9247 ± 0.0

[†] F1*, Precision* and Recall* refer to the metrics calculated using the point-adjusted evaluation strategy.

that minimize both false positives and false negatives. This design ensures superior F1 performance, providing a robust solution for detecting deviations in PLC behavior.

VI. RELATED WORK AND LIMITATIONS

A. Logic Recovery

Logic recovery aims to infer the underlying PLC logic solely from input-output data. This concept overlaps with system identification and symbolic regression, as all three approaches focus on deriving mathematical representations from observed data. In this section, we review these related methods to contextualize our approach.

1) *System Identification*: System identification (SI) has been a prominent research interest (see [39] and references therein), evolving from the identification of linear systems to encompass non-linear and hybrid system identification. Classical SI methods like ARX, FIR, and ARMAX employ linear combinations of inputs to predict system outputs, and this paradigm extends to nonlinear systems in methods such as NARX [9] and NARMAX [40]. Some SI methods detect multi-mode systems by fine-tuning the parameters of a predefined nonlinear model, such as the linear piecewise affine system identification method [41] and the hybrid system identification method [42]. In comparison with these methods, SRLR offers greater flexibility, as it does not require prior knowledge of the model structure.

2) *Symbolic Regression*: Existing symbolic regression methods can be broadly categorized into three groups: heuristic methods, pre-trained methods, and reinforcement learning methods. Heuristic methods [10], [11] employ evolutionary techniques like GP to explore the search space for optimal solutions. However, these methods face scalability issues and are sensitive to hyperparameters [16]. Pre-trained methods [43], [44] leverage pre-trained models to provide prior knowledge for guiding the initial search space exploration. However, these methods primarily focus on numerical results, making it challenging to handle symbols due to their discrete characteristics. Reinforcement learning methods [16], [45], on the other hand, sample symbols and address the discrete sampling process using policy gradient training. This approach has proven to be efficient and has achieved SOTA results in symbolic regression benchmarks. Thus, we adopt DSR [16] in

this work as the backbone framework for identifying control logic.

3) *Logic Recovery*: The concept of logic recovery has been explored extensively [9], [39]. For instance, Schmidt and Lipson [10] employed an evolutionary-based symbolic regression method to uncover physical laws through motion-tracking data, striking a balance between computational complexity and accuracy of discovery with the Pareto Front. Ly et al. [12] proposed the Cluster SR designed to identify time-invariant nonlinear hybrid systems. They integrated GP and the Expectation Maximization (EM) algorithm to infer the equations governing each mode and introduced a classification method to recover the transition logic among distinct modes within the hybrid system. In our experiments, existing clustering-based methods performed poorly in identifying complex nonlinear PLC logics. For example, mode memberships determined by Cluster SR were often sparse and inaccurate in complex systems, largely due to their reliance on soft assignments and the absence of continuity constraints. By contrast, SRLR achieved a 39% improvement in BFR in noisy environments compared with Cluster SR. Furthermore, without key designs used in SRLR like the s-domain enhancement and complexity regularization, existing symbolic regression methods such as PySR and Operon failed to identify subsystems in load frequency control systems and automatic voltage regulators, including PID controllers.

Several studies have explored methods for reverse-engineering PLC logic. Reditus [46] captures network packets generated during PLC program execution and reconstructs the logic using the decompilation tools provided by PLC workstations. However, this approach is limited to a small subset of PLCs; for example, recent platforms such as Siemens TIA Portal and Codesys do not support reverse-engineering functions. Another line of research proposes extracting PLC logic from compiled binaries through database mapping. In this method [47], a database maps binary structures to their corresponding control program components, and similarity analysis is used to match target code segments. While effective in some cases, it is impractical to enumerate all mapping relationships needed to reconstruct complex control logic, especially those involving varied arithmetic operations. CLEVER [48] addresses this challenge by analyzing PLC control applications directly. Using heuristic algorithms, it

identifies variable dependencies and sequentially decompiles the source code. However, its effectiveness is limited when variable dependencies are incomplete, leading to failures in recognizing certain numerical operations, such as cyclic shift functions in motor control programs.

In contrast, SRLR demonstrates strong capabilities in handling numerical calculations. First, it employs s-domain modeling to broaden the scope of recognized operations, enabling the modeling of motor logic in LFC systems. Second, SRLR focuses on recovering mathematical relationships between variables, making the precise code implementation less critical in some contexts. These strengths allow SRLR to operate independently of specific programming platforms while offering robust logic reconstruction capabilities.

B. Attack Detection

Attack detection methods in ICSs are crucial for addressing threats in both cyber and physical layers and these methods can be broadly categorized into specification-based and ML-based approaches. Specification-based methods [11], [13] derive rules from a database of known attacks and employ these rules to identify threats as they occur. These methods typically offer high accuracy and clear explanations; however, their major limitation lies in the difficulty of detecting zero-day attacks, which are not represented in the existing database. In contrast, ML-based methods [4], [5], [14], [15] leverage statistical or machine learning techniques to develop patterns of normal data, which facilitates the detection of zero-day attacks by recognizing patterns that deviate from the norm. Despite this capability, ML-based methods often suffer from a low true positive rate, particularly when attack data falls within the normal data range or does not significantly deviate from established patterns. Additionally, ML-based methods cannot generally provide clear, easily understandable explanations for their detection decisions. From the perspective of ICS security, related efforts can be broadly categorized into three areas: process-level protection, sensor data integrity, and PLC code integrity verification. In this section, we review representative work in each of these areas and discuss relevant AI explainability methods that related to the goals of our approach.

1) *Process-Level Protection*: Regarding process control attacks, Hadziosmanovi et.al [49] proposed monitoring the status of PLC variables and developing a regression model to predict variable traces, aiming to detect discrepancies that do not match the predicted patterns. However, in modern ICSs, PLC outputs are predominantly binary variables, such as commands for controlling valves or pumps in water treatment systems. This characteristic poses a significant challenge for time-series models, as accurately predicting binary outcomes often proves difficult. Tan et al. [8] introduced CoToRu, a method for inferring the behavior of PLCs to detect attacks. Their approach involves parsing the PLC code to generate a state transition table, which is then used to derive rules for detecting threats. While their method is effective and achieves high accuracy, obtaining access to the PLC code can be challenging for third-party security teams in real-world scenarios.

2) *Sensor Data Integrity*: Numerous studies have focused on protecting sensor data integrity. A common approach involves using prediction models to forecast future sensor values. PyCRA [50], for example, enhances these prediction methods by comparing current readings with historical data, employing a noise-reduction strategy to detect stealthy integrity attacks. Another widely-used method is the Linear Dynamical State-Space model, which simulates process behavior using known signals and compares the simulated values with sensor response. While effective for process-level data integrity detection, these methods struggle with predicting the outputs of controllers. SRLR, on the other hand, is specifically designed to counter logic attacks on controllers. Unlike the methods above, which focus on sensor data integrity, SRLR addresses the core logic of controllers, making it a complementary and orthogonal approach. Deploying SRLR alongside sensor data integrity protection methods can provide comprehensive protection against both process-level attacks (such as false data injection via sensor) and control logic tampering.

3) *Attesting PLC Code Integrity*: Attesting code integrity is another potential solution for addressing PLC logic tampering. This can be achieved through root-of-trust mechanisms in specialized hardware modules or checksum verification via updating PLC software. However, these methods often require invasive hardware modifications or PLC vendor's support [51] and are vulnerable to time-of-check-to-time-of-use (TOCTOU) attacks [52]. In contrast, SRLR can be integrated into existing PLC I/O monitoring platforms, such as commercial-ready level 0 monitoring solutions,² to offer advanced logic attack detection in a non-invasive manner.

C. Scope and Limitations

PLC logic covers a wide range of control strategies. From a control theory perspective, these strategies fall into two broad categories: open-loop control, such as simple switches, and closed-loop control, including PID controllers. Our experiments show that the SRLR framework effectively recovers certain open-loop behaviors, such as hysteresis relay logic, and accommodates variations of PID control commonly used in closed-loop systems. However, SRLR has limitations when applied to complex control strategies such as Model Predictive Control and $\mathcal{H}_\infty$ control. These approaches involve solving quadratic programming or semi-definite programming problems with intricate constraints, making it difficult to represent their logic through simple mathematical expressions. As a result, such advanced control methods fall outside the current scope of SRLR.

From a syntactic standpoint, PLC programs rely on a set of standardized function blocks, which can be grouped into key categories: Bitwise operations; Timers; Counters; Edge detection; Data operations; Control functions. Many of these constructs—such as bitwise logic and arithmetic—map directly to mathematical symbols, expanding the SRLR framework's symbolic library, enabling effective handling of

²<https://sigasec.com>

such logic. Timer functions can also be expressed mathematically. For example, the TON (on-delay) timer activates when its input signal remains high for a duration longer than a predefined threshold. This behavior can be represented as:

$$Q(t) = H \left(I(t)\Delta t \sum_{\tau=0}^N I(t-\tau) - PT \right), \quad (22)$$

where $I(t)$ is the input signal, PT the preset time, Δt the PLC scan cycle, and H the Heaviside step function. The accumulated time $T_{acc}(t) = I(t)\Delta t \sum_{\tau=0}^N I(t-\tau)$, similar to an EWMA, is reset whenever the input is low. SRLR has been shown to recover EWMA-like patterns even in noisy environments, supporting its ability to model timer behaviors.

Other PLC structures can be similarly translated into mathematical representations, such as counters and edge triggers. These results collectively demonstrate that SRLR is capable of capturing basic PLC syntax. However, the mathematical modeling of more complex process control and sequential structures like interrupt and asynchronous event handling becomes significantly more complicated. These advanced control logics are currently beyond the scope of SRLR.

While SRLR shows promising results in our experiments across various ICS domains, thorough evaluation with a more diverse set of control logic is needed to characterize SRLR's capability and applicability — specifically, future work is needed to answer the following questions: which type of control logic SRLR can handle well, and which types it cannot? Is there a complexity bound for the control logic, beyond which, SRLR can no longer work reliably? If that is the case, can we find a reliable way to divide and conquer a complicated system using SRLR? We hope our work can inspire future research in these directions.

VII. CONCLUSION

We presented SRLR, a novel approach to recover logic of PLC and generate rules for detecting logic manipulation attacks in ICSs. SRLR accurately infers formulas for both single-mode and multi-mode processes, recovering the control logic of PLCs. The recovered logic helps generate rules capable of detecting attacks such as controller configuration tampering, malicious controller output injection, and controller disabling. SRLR is tested on both a grid frequency control system and a water treatment testbed. Results indicate that SRLR outperforms existing deep anomaly detection models, while providing explainability to its decisions. In future work, we will evaluate SRLR under more complex control logic and determine the applicable range of such methods.

ACKNOWLEDGMENT

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore. The work of Hao Zhou was completed when he was with Singapore University of Technology and Design (SUTD).

REFERENCES

- [1] J. Giraldo et al., "A survey of physics-based attack detection in cyber-physical systems," *ACM Comput. Surveys*, vol. 51, no. 4, pp. 1–36, Jul. 2019.
- [2] H. Zhu, M. Liu, B. Chen, X. Che, P. Cheng, and R. Deng, "HoneyJudge: A PLC honeypot identification framework based on device memory testing," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 6028–6043, 2024.
- [3] D. Kushner, "The real story of stuxnet," *IEEE Spectr.*, vol. 50, no. 3, pp. 48–53, Mar. 2013.
- [4] C. Feng and P. Tian, "Time series anomaly detection for cyber-physical systems via neural system identification and Bayesian filtering," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 2858–2867.
- [5] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series data," 2022, *arXiv:2201.07284*.
- [6] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 4027–4035.
- [7] I. Stellios, P. Kotzanikolaou, M. Psarakis, C. Alcaraz, and J. Lopez, "A survey of IoT-enabled cyberattacks: Assessing attack paths to critical infrastructures and services," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 3453–3495, 2018.
- [8] H. C. Tan, C. Cheh, and B. Chen, "CoToRu: Automatic generation of network intrusion detection rules from code," in *Proc. IEEE Conf. Comput. Commun.*, May 2022, pp. 720–729.
- [9] L. Ljung, C. R. Andersson, K. Tiels, and T. B. Schön, "Deep learning and system identification," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 1175–1181, 2020.
- [10] M. Schmidt and H. Lipson, "Distilling free-form natural laws from experimental data," *Science*, vol. 324, no. 5923, pp. 81–85, Apr. 2009.
- [11] T. N. Mundhenk, M. Landajuela, R. Glatt, C. P. Santiago, D. M. Faissol, and B. K. Petersen, "Symbolic regression via neural-guided genetic programming population seeding," 2021, *arXiv:2111.00053*.
- [12] D. L. Ly and H. Lipson, "Learning symbolic representations of hybrid dynamical systems," *J. Mach. Learn. Res.*, vol. 13, no. 1, pp. 3585–3618, 2012.
- [13] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, Dec. 2019.
- [14] S. Han and S. S. Woo, "Learning sparse latent graph representations for anomaly detection in multivariate time series," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 2977–2986.
- [15] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "Usad: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 3395–3404.
- [16] B. K. Petersen, M. Landajuela, T. N. Mundhenk, C. Santiago, S. Kim, and J. T. Kim, "Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [17] J. Goh, S. Adepu, K. N. Junejo, and A. P. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. 11th Int. Conf. Crit. Inf. Infrastructures Security*, Oct. 2017, pp. 88–99.
- [18] H. Wang et al., "Scientific discovery in the age of artificial intelligence," *Nature*, vol. 620, no. 7972, pp. 47–60, 2023.
- [19] R. Spenneberg, M. Brüggemann, and H. Schwartke, "PLC-blaster: A worm living solely in the PLC," *Black Hat Asia*, vol. 16, pp. 1–16, Mar. 2016.
- [20] W. Aoudi, M. Iturbe, and M. Almgren, "Truth will out: Departure-based process-level detection of stealthy attacks on control systems," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 817–831.
- [21] S. Fuller, B. Greiner, J. Moore, R. Murray, R. van Paassen, and R. Yorke, "The Python control systems library (Python-control)," in *Proc. 60th IEEE Conf. Decis. Control (CDC)*, Dec. 2021, pp. 4875–4881.
- [22] M. Kumar, B. Packer, and D. Koller, "Self-paced learning for latent variable models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 23, 2010, pp. 1189–1197. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2010/file/e57c6b956a6521b28495f2886ca0977a-Paper.pdf
- [23] H. Akaike, "A new look at the statistical model identification," *IEEE Trans. Autom. Control*, vol. AC-19, no. 6, pp. 716–723, Dec. 1974.

- [24] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 387–395.
- [25] D. Masti and A. Bemporad, "Learning nonlinear state–space models using autoencoders," *Automatica*, vol. 129, Jul. 2021, Art. no. 109666.
- [26] K. B. Newhart, R. W. Holloway, A. S. Hering, and T. Y. Cath, "Data-driven performance analyses of wastewater treatment plants: A review," *Water Res.*, vol. 157, pp. 498–513, Jun. 2019.
- [27] M. Cranmer, "Interpretable machine learning for science with PySR and SymbolicRegression.Jl," 2023, *arXiv:2305.01582*.
- [28] B. Burlacu, G. Kronberger, and M. Kommenda, "Operon C++: An efficient genetic programming framework for symbolic regression," in *Proc. Genetic Evol. Comput. Conf. Companion*, Jul. 2020, pp. 1562–1570.
- [29] H. Saadat, *Power System Analysis*, vol. 2. New York, NY, USA: McGraw-Hill, 1999.
- [30] Z. A. Obaid, L. M. Cipcigan, and M. T. Muhssin, "Fuzzy hierarchal approach-based optimal frequency control in the great Britain power system," *Electric Power Syst. Res.*, vol. 141, pp. 529–537, Dec. 2016.
- [31] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 2828–2837.
- [32] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "Lstm-based encoder–decoder for multi-sensor anomaly detection," 2016, *arXiv:1607.00148*.
- [33] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Represent.*, Feb. 2018, pp. 1–19. [Online]. Available: <https://openreview.net/forum?id=BJJLHbb0>
- [34] H.-B. Shin and J.-G. Park, "Anti-windup PID controller with integral state predictor for variable-speed motor drives," *IEEE Trans. Ind. Electron.*, vol. 59, no. 3, pp. 1509–1516, Mar. 2012.
- [35] L. R. da Silva, R. C. Flesch, and J. E. Normey-Rico, "Analysis of anti-windup techniques in pid control of processes with measurement noise," *IFAC-PapersOnLine*, vol. 51, no. 4, pp. 948–953, 2018.
- [36] L. Jia and X. Zhao, "An improved particle swarm optimization (PSO) optimized integral separation PID and its application on central position control system," *IEEE Sensors J.*, vol. 19, no. 16, pp. 7064–7071, Aug. 2019.
- [37] J. Liu, Y. Gu, L. Zha, Y. Liu, and J. Cao, "Event-triggered H_∞ load frequency control for multiarea power systems under hybrid cyber attacks," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 49, no. 8, pp. 1665–1678, Aug. 2019.
- [38] D. EStimator, "A fast algorithm for the minimum covariance," *Technometrics*, vol. 41, no. 3, p. 212, 1999.
- [39] A. P. Sage, "Perspectives on system identification," *IFAC Proc. Volumes*, vol. 17, no. 2, pp. 1–12, Jul. 1984.
- [40] S. A. Billings, *Nonlinear System Identification: NARMAX Methods in the Time, Frequency, and Spatio-temporal Domains*. Hoboken, NJ, USA: Wiley, 2013.
- [41] L. Bako, "Identification of switched linear systems via sparse optimization," *Automatica*, vol. 47, no. 4, pp. 668–677, Apr. 2011.
- [42] Y. Yuan et al., "Data driven discovery of cyber physical systems," *Nature Commun.*, vol. 10, no. 1, p. 4894, 2019.
- [43] S. Holt, Z. Qian, and M. van der Schaar, "Deep generative symbolic regression," 2023, *arXiv:2401.00282*.
- [44] M. Valipour, B. You, M. Panju, and A. Ghodsi, "Symbolicgpt: A generative transformer model for symbolic regression," 2021, *arXiv:2106.14131*.
- [45] M. Landajuela et al., "Discovering symbolic policies with deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 5979–5989.
- [46] S. A. Qasim, J. M. Smith, and I. Ahmed, "Control logic forensics framework using built-in decompiler of engineering software in industrial control systems," *Forensic Sci. International: Digit. Invest.*, vol. 33, Jul. 2020, Art. no. 301013.
- [47] S. Kalle, N. Ameen, H. Yoo, and I. Ahmed, "CLIK on PLCs! Attacking control logic with decompilation and virtual PLC," in *Proc. Binary Anal. Res. Workshop, Netw. Distrib. Syst. Secur. Symp.*, Feb. 2019, pp. 1–12.
- [48] C. Sang, J. Wu, J. Li, and M. Guizani, "From control application to control logic: PLC decompile framework for industrial control system," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 8685–8700, 2024.
- [49] D. Hadziosmanović, R. Sommer, E. Zambon, and P. H. Hartel, "Through the eye of the PLC: Semantic security monitoring for industrial processes," in *Proc. 30th Annu. Comput. Security Appl. Conf.*, 2014, pp. 126–135.
- [50] Y. Shoukry, P. Martin, Y. Yona, S. Diggavi, and M. Srivastava, "PyCRA: Physical challenge-response authentication for active sensors under spoofing attacks," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2015, pp. 1004–1015.
- [51] W. Lin, H. C. Tan, B. Chen, and F. Zhang, "DNAttest: Digital-twin-based non-intrusive attestation under transient uncertainty," in *Proc. 53rd Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*. Bedford, MA, USA: Digital, Jun. 2023, pp. 376–388.
- [52] S. Bratus, N. A. D'Cunha, E. R. Sparks, and S. W. Smith, "TOCTOU, traps, and trusted computing," in *Proc. Int. Conf. Trusted Comput.*, 2008, pp. 14–32.



Hao Zhou received the B.Sc. degree in communication engineering from Wuhan University of Technology in 2019 and the Ph.D. degree in information and communication engineering from Beijing University of Posts and Telecommunications in 2025. He was a Visiting Ph.D. Student with Singapore University of Technology and Design from 2023 to 2024. His current research interests include AI-native radio access networks and time series modeling for forecasting and anomaly detection.



Suman Sourav received the Ph.D. degree from the National University of Singapore. He is currently an Assistant Professor with Aalborg University, Denmark. He was previously a Research Fellow with Singapore University of Technology and Design and Illinois Advanced Research Centre, Singapore. His research interests include the interplay of theory and application, considering both theoretical as well as practically implemented algorithmic aspects of real-world systems, including topics such as satellite networks, edge-cloud systems, distributed computing, and cyber-physical systems. He received the Research Achievement Award from the National University of Singapore.



Binbin Chen (Member, IEEE) received the B.Sc. degree in computer science from Peking University in 2003 and the Ph.D. degree in computer science from the National University of Singapore in 2010. Since July 2019, he has been an Associate Professor with the Information Systems Technology and Design (ISTD) Pillar, Singapore University of Technology and Design (SUTD). His current research interests include wireless networks, cyber-physical systems, and cyber security for critical infrastructures.



Ke Yu received the B.S. degree in computer science and the Ph.D. degree in signal and information processing from Beijing University of Posts and Telecommunications (BUPT), China, in 2000 and 2005, respectively. She was a Visiting Researcher with the University of Agder, Norway, in 2011. From 2015 to 2016, she was a Visiting Scholar with the University of Illinois at Chicago, USA. She is currently a Professor with the School of Artificial Intelligence, BUPT. Her research interests include communication network theory, network data mining and anomaly detection, and internet of agents.